Andrés Parejo Domínguez

u6103252

HYPERMEDIA PROJECT, PT. 2: Documentation

Project analysis:

1. Interface and interaction

First of all, my idea was to create an interface to provide a simple, intuitive and user-friendly experience in the game. For that reason, the design follows a minimalist approach, using a consistent colour palette and clearly separated sections.

I divided the website into three main screens: the main menu, the game screen and the victory screen.

The main menu is the first screen displayed when the website is opened. Basically, it contains the game title, a short list of instructions and the level selection area. In my case, I designed five increasingly difficult levels for the user (Easy, Medium, Difficult, Expert and Impossible), which are represented by clickable cards containing a miniature preview of the initial board configuration of each level. These previews allow players to get a visual impression of each level before starting it and help to give the user an idea of its difficulty. Therefore, the level selection is more intuitive and gives the menu a more interactive appearance.

The following step was creating the game screen, which appears after a level is selected. This screen is divided into three sections. At the top, we can see the game information, where I displayed the current level name, a button to return to the main menu and a move counter. Moreover, the central element is a 6x6 grid that contains all the cars. The main car is highlighted in green, while the other ones use different colours and letters to make them easier to distinguish. The board has the exit opening on the right side.

Finally, in the game controls section, the player can restart the current level, undo the previous movement or use directional buttons to move the selected car. The player has the option to drag and drop the selected car in the permitted directions or use the arrow controls to move it one cell at a time (the arrows automatically adapt to the orientation of the selected car, horizontally or vertically).

On the other hand, I also added elements to make the game more intuitive. For instance, the selected car is highlighted with a darker outline, and a message below the board indicates which vehicle is currently selected, which provides visual feedback and helps the player understand which car will be affected by the controls.

As well as that, the move counter displays the number of movements performed during the current game. These movements can be modified by the undo button (which allows players to reverse their previous movement) and by the restart button, which restores the original configuration of the level.

When the user completes the level (when the main car reaches the exit), they are redirected to the victory screen. Basically, a congratulatory message informs the player that the level has been completed and displays the total number of moves required. Afterwards, the player can decide whether they want to play the next level, play the same one again or go back to the main menu. All these options allow the user to continue playing without having to reload the page.

As I said before, the visual design that I implemented uses a light background combined with green tones to highlight important elements, especially the main car and the interactive controls. The other vehicles use different soft colours to improve visual distinction and clearly distinguish the main car. Moreover, I maintained the same typography (Times New Roman), colours and button styles throughout the application to ensure visual consistency, and I combined different font sizes and weights to provide a good visual hierarchy for the website. Another good detail is that the interface also includes hover effects on interactive elements and disabled states for controls that cannot currently be used, which can be useful to make the user aware of which elements they can select. Finally, I added media queries to adapt the layout to smaller screens.

To sum up, I believe that my website is able to keep the interface clean and understandable, while allowing the player to focus on the main purpose: solving the puzzle.

2. Implementation

I developed the game by dividing the implementation into three separate files: index.html (structure), styles.css (appearance) and script.js (interactions).

In the first place, the HTML file defines the main structure of the website using elements such as <main>, <section>, , , etc. As I said before, the structure of the website is separated into the menu screen, which contains the title, instructions and level selection cards, the game screen (game information, board and interaction controls), and a victory screen in order to display the congratulatory message and the available options after winning the game. Obviously, only one screen is visible at a time, while the other ones are hidden (through CSS rules). As well as that, the game board itself is empty in the HTML file, because its cells and cars are generated dynamically using JavaScript.

Secondly, thanks to CSS, we can achieve a good visual design and position all the game elements. First of all, the variables are defined at the beginning of the file in order to use them later to set the main colours and typography. I feel that this makes it easier to maintain a consistent appearance and modify the design.

I also used Grid and Flexbox properties to organise the available levels into cards in the level selection area and, in other sections, to align buttons and interface elements. The game board is also implemented using Grid properties (with six rows and six columns). Moreover, the cars use absolute positioning to allow JavaScript to update their dimensions according to their orientation and position.

For the rest of the file, I used different classes to set the necessary rules and distinguish horizontal and vertical vehicles, the main car, the selected one, the different vehicle colours, etc. Finally, I also included hover effects, button states and the media query to improve the overall user experience.

The JavaScript file is the most important one, because it contains the main functionality of the application.

The first step was defining the constants that I was going to use. GRID_SIZE defines the dimensions of the board (6x6), the COLORS array contains the CSS classes used to assign different colours to the cars, while the LEVELS array stores the initial configuration of all five difficulty levels. Each car is represented by an object containing the ID of the vehicle, its initial position (row and column), its length (number of cells), its orientation (vertical or horizontal) and a boolean value to identify which one is the main car. This structure makes it possible to add or modify levels without changing the main game logic.

After that, I defined all the references to DOM elements that I was going to use later, and I also initialised several variables used to keep track of the current game, such as the index of the current level, the drag state, the move history and whether the game has finished or not.

Then, practically the whole file is based on the implementation of the different functions used in the application in order to make the game work. For instance, the first one creates independent copies of the original car configurations to prevent movements from modifying the original level definitions. It also allows levels to be restarted correctly.

Afterwards, I defined the functions to display the different elements on the screen. The showScreen() function controls which section of the website is visible, the showMenu() function generates the level selection cards with the corresponding

previews, and startLevel() initialises the selected level (with the game state, copies of the original car positions, clears the movement history and displays the game screen).

Alongside these functions, we have the ones used to generate the board and position the vehicles. Basically, showBoard() dynamically generates the 36 cells of the board and creates HTML elements representing each car. It is also used to calculate the dimensions and positions of the vehicles and update the CSS properties to display each car in the right position. On the other hand, the updateExit() function positions the exit opening at the same row as the main vehicle.

Then, I implemented some functions that are responsible for validating the movement and direction of the different cars and ensuring that the game follows its rules (for instance, that a car cannot go beyond the limits of the board):

- **getInitPosition():** returns the initial coordinate along the car's movement
- **getCarCells():** calculates the board cells occupied by a car
- **isInsideBoard():** checks whether a coordinate belongs to the board
- **freeCell():** checks whether a cell is available and not occupied by another car
- **canPlaceAt():** verifies whether a car can be placed at a new position without leaving the board or colliding with another car

Furthermore, the moveCar() function is used to execute car movements. Basically, it receives the vehicle ID and the requested movement distance. Then, before updating its position, we call the canPlaceAt() function to check the available cells one at a time, stopping when it reaches an obstacle or the board boundary. As a result, if the car moves successfully, its previous position is saved in the moveHistory variable (also, we have to update the board and game information and check whether the player has completed the level).

I implemented the extra functionalities (undo and restart) with the undoMove() and resetLevel() functions. The first one retrieves the last movement stored in moveHistory and restores the corresponding vehicle to its previous position, while the second one restarts the current level by calling startLevel() again and resetting all the data. As well as that, I implemented the dragging system using different mouse events in functions to detect the different positions and states of the mouse. Mainly, the following ones are: carPointerDown(), carPointerMove() and carPointerUp(). As a detail, these functions also allow a dragging operation to continue even when the cursor temporarily moves outside the board.

Finally, the checkVictory() function verifies whether the main vehicle has reached the right edge of the board, where the exit is located. Then, when the objective is achieved, the gameFinished variable is set to true (preventing further movements), the

victory message is updated with the total number of moves, and the victory screen is displayed.

As well, towards the end of the JavaScript file, I declared event listeners to connect the interface controls to their corresponding functions, which include the level selection buttons, directional buttons, undo and restart buttons, navigation options and victory screen controls. There is also a resize event listener to redraw the board when the browser window changes size.

Overall, I feel that my implementation provides the right Rush Hour structure and gameplay experience. From my point of view, the implementation in three separate files and the separation between the interface, visual design and game logic also make it easier to understand, maintain and extend with additional levels or features.